Software Requirements Specification (SRS Documentation)

Table of Contents

1. Introduction

1.1 Purpose

This document defines the functional and non-functional requirements for an advanced Retrieval-Augmented Generation (RAG) system for cybersecurity education, leveraging modern LLMs (such as OpenAI GPT) and domain-specific retrieval, reranking, and accuracy evaluation tools. The primary purpose is to create an interactive educational platform that assists users, for example cybersecurity students, of varying expertise levels in understanding cybersecurity threats, vulnerabilities, defense mechanisms, and industry best practices.

The enhanced system does this by providing interactive, accurate guidance for learners and professionals, combining curated cybersecurity knowledge bases (e.g., OWASP) and state-of-the-art vector search, evaluation, and visualization

1.2 Document Conventions

- **Shall**: Indicate mandatory requirements that must be implemented
- **Should**: Indicate desirable features that are important but not critical
- **May**: Indicate optional features that would enhance the system and improve performance
- **RAG**: Retrieval-Augmented Generation
- **LLM**: Large Language Model
- **OWASP**: Open Web Application Security Project
- **API**: Application Programming Interface
- **NLP**: Natural Language Processing
- **UI**: User Interface
- **KB**: Knowledge Base
- **Sentence Transformers**: Embedding library
- **pdfplumber/pydf**: PDF handling libraries
- **Streamlit**: Python web Framework
- **BeautifulSoup4/lxml**: HTML/text parsing (supported)
- **Cross-Encoder**: Document reranking libraries

1.3 Intended Audience

This document is intended for:

- **End Users**: Cybersecurity students, educators, and cybersecurity professionals (for requirement validation)
- **Project Stakeholders**: Faculty supervisors and academic committee members
- **Future Maintainers**: Developers who will extend or modify the RAG system

1.4 Project Scope

This project delivers a complete RAG-powered educational platform with:

- Retrieval of cybersecurity knowledge (OWASP, custom KB)
- LLM-based response generation
- Streamlit web UI
- Evaluation dashboards comparing RAG vs non-RAG performance

In Scope:

- Process natural language queries about cybersecurity topics
- Retrieve relevant information from curated OWASP and cybersecurity knowledge bases using [sentence-transformers]
- Generate educational responses using OpenAI's ChatGPT with retrieved context
- Allow for performance and grounding metric visualization (token overlap, bigram, sentence attribution)
- Provide interactive web interface via Streamlit
- Implement vector-based document retrieval using ChromaDB
- Automated PDF to text ingestion via [pdfplumber]
- Secure API key handling
- Integrated reranking of retrieved chunks with [cross-encoder/ms-marco-MiniLM-L-12-v2]

Out of Scope:

- Real-time threat intelligence integration
- Multi-language support beyond English
- Advanced user authentication and authorization systems beyond basic environment variable API keys
- Integration with external learning management systems (LMS)
- Mobile application development

1.5 References

1. OWASP Top 10 Documentation
2. OpenAI API Documentation
3. ChromaDB Documentation
4. Streamlit Deployment Tutorials
5. pdfplumber & pypdf Official Docs
6. Sentence Transformers & Cross-Encoder Docs
7. IEEE Standard 830-1998 for Software Requirements Specifications
8. NIST Cybersecurity Framework v1.1

2. Overall Description

2.1 Product Perspective

The Cybersecurity Education RAG System is now a **Streamlit web application** with modular backend components:

System Architecture Components:

- Document conversion: Automated PDF to text

- Knowledge base chunking and embedding with [sentence-transformers]
- Semantic retrieval and reranking (hybrid pipeline with Cross-Encoder)
- Response generation via OpenAI LLMs
- Streaming UI and metrics dashboard (Streamlit + plotly)
- Accuracy/grounding metrics for every answer
- Easy extensibility for new data or model upgrades

External Dependencies:

- OpenAI API (Chat Completion endpoint)
- Internet connectivity for embeddings and model inference

System Boundaries: The system interfaces with external services (OpenAI API) while maintaining local data processing and storage capabilities. It operates independently without requiring integration into existing enterprise systems, making it suitable for educational and research environments.

2.2 Product Features

The system provides the following major features:

1. Natural Language Query Understanding with context expansion
2. **Advanced Document Retrieval**:
    - Vector embeddings, similarity search
    - Reranking of top results using Cross-Encoder
    - Configurable chunk size and document filters
3. **Intelligent Response Generation**:
    - Uses both user query and retrieved context
    - Supports RAG and ChatGPT-only answer workflows
4. **Knowledge Base Management**:
    - Automated ingestion (PDF, TXT, HTML)
    - Duplicate detection, integrity checks

5. **Evaluate and Visualize**:
    - Grounding metrics for every answer
    - Side-by-side metric tables and interactive dashboards
6. **User Interface**:
    - Streamlit-based input/query panels
    - Context transparency (see supporting docs for each answer)
    - Exportable/comparable metrics

2.3 User Classes and Characteristics

Primary Users:

1. **Cybersecurity Students (Novice Level)**
    - Characteristics: Limited cybersecurity knowledge, learning fundamental concepts
    - Usage Pattern: Frequent queries about basic concepts, definitions, and examples
    - Technical Expertise: Low to medium
    - System Interaction: Guided learning, step-by-step explanations needed
2. **Cybersecurity Educators (Intermediate Level)**
    - Characteristics: Teaching cybersecurity courses, need comprehensive explanations
    - Usage Pattern: Complex queries about teaching materials and detailed explanations
    - Technical Expertise: Medium to high
    - System Interaction: Detailed responses, educational context required
3. **Cybersecurity Professionals (Advanced Level)**
    - Characteristics: Working in the field, need quick reference and advanced topics
    - Usage Pattern: Specific technical queries, compliance requirements, best practices
    - Technical Expertise: High
    - System Interaction: Concise, technical responses with industry standards

Secondary Users:

4. **Researchers and Academics**
    - Characteristics: Conducting cybersecurity research, need comprehensive information
    - Usage Pattern: In-depth queries about latest trends and methodologies
    - Technical Expertise: High
    - System Interaction: Detailed, research-oriented responses

2.4 Operating Environment

- OS: Windows, Linux, macOS
- Python 3.8+
- Libraries: streamlit, openai, sentence-transformers, torch, plotly, python-dotenv, pdfplumber, pypdf, beautifulsoup4, lxml, numpy
- Hardware: ≥4GB RAM, modern CPU, broadband internet
- API Access: OpenAI key required

2.5 Design and Implementation Constraints

Technical Constraints:

1. **API Rate Limits**: OpenAI API usage limits may restrict concurrent users and query frequency
2. **Model Dependencies**: All external libraries under compatible open-source licenses
3. **Local Storage**: Local disk for KB + embeddings
4. **Internet Dependency**: Critical functionality requires stable internet connection
5. **System uses modular file structure**: new chunks/sources easy to add

Regulatory Constraints:

1. **Data Privacy**: Must comply with educational data privacy requirements
2. **API Terms of Service**: Must adhere to OpenAI usage policies and terms
3. **Open-Source Licensing**: Compliance with open-source library licenses

Performance Constraints:

1. **Response Time**: System must respond to queries within 10 seconds under normal conditions
2. **Concurrent Users**: Initial version supports single-user interaction
3. **Memory Usage**: System should operate efficiently within 2GB RAM allocation
4. **Storage Growth**: Knowledge base expansion must be manageable within storage constraints

2.6 User Documentation

The system shall include the following documentation:

- **Installation Guide:** Python/version setup, requirements install
- **User Manual:** Query, comparison, and UI walkthroughs
- **Troubleshooting:** Error resolution, API/encoding issues
- **Contributor Guide:** Steps for new KB sources or model config
- **FAQ:** Pipeline, privacy, optimization

2.7 Assumptions and Dependencies

Assumptions:

1. Users have basic computer literacy and command-line familiarity
2. Internet connectivity is stable and reliable during system usage
3. OpenAI API service remains available and maintains current pricing structure
4. Users require responses in English language only
5. Educational use case does not require real-time threat intelligence
6. Knowledge base content is high quality and properly curated
7. API key stored in .env
8. Data sources pre-exist in supported formats (.txt, .pdf, .html)
9. Python environment correctly configured (Python/pip available)

External Dependencies:

1. **OpenAI API**: Critical dependency for response generation
2. **ChromaDB**: Vector database for document storage and retrieval
3. **Sentence Transformers**: Embedding model for semantic search
4. **Python Ecosystem**: Various libraries for data processing and API interactions

Internal Dependencies:

1. **Knowledge Base Curation**: Quality and coverage of educational content
2. **Prompt Engineering**: Effectiveness of prompt templates for different query types
3. **System Integration**: Proper integration between retrieval and generation components
4. **Performance Optimization**: Efficient processing of queries and responses

3. System Features

3.1 Intelligent Query Processing and Response Generation

Feature Description

Processes user cybersecurity questions and produces context-grounded answers using retrieval + LLM.

Functional Requirements

REQ-3.1.1 The system shall accept natural-language cybersecurity questions.

REQ-3.1.2 The system shall generate responses within 10 seconds for 95% of queries.

REQ-3.1.3 Responses shall include:

- Explanation
- Mitigation techniques
- Standards and references

REQ-3.1.4 Conversation state shall be maintained using Streamlit session state.

REQ-3.1.5 The system shall support:

- Ask with RAG mode
- Ask ChatGPT Only mode
- Comparison mode

REQ-3.1.6 The system shall compute these metrics:

- Token Overlap %
- Bigram F1
- Sentence Attribution %

REQ-3.1.7 The system shall display the results in a table.

New Features:

- Grounding metrics (token overlap, bigram F1, sentence attribution) computed for each response
- Ability to compare RAG answers with ChatGPT-only answers directly in the UI
- PDF-to-text integration for knowledge base expansion
- Visualization with plotly gauges and comparative tables

3.2 Knowledge Base Management and Retrieval

REQ-3.2.1: Automatic ingestion of OWASP, structured by category and risk

REQ-3.2.2: Chunking and vector embedding with sentence-transformers

REQ-3.2.3: Retrieval using semantic search + reranking (Cross-Encoder)

REQ-3.2.4: Configurable retrieval params (docs returned, thresholds)

REQ-3.2.5: Expansion via PDF or custom content ingestion

REQ-3.2.6: Content statistics, performance analytics, knowledge gap ID

New Features:

- PDF support for bulk KB ingest
- Optional HTML/text parsing for advanced sources
- Duplicate and integrity checks in ingest routine
- Storage and stats on chunk coverage and retrieval efficiency
- Exportable and batch analytics for KB quality assessment

4. External Interface Requirements

4.1 User Interfaces

The main user interface is Streamlit UI. The design follows modern UI and UX principles, with a focus on clarity and simple navigation.

Features:

- Navigation header and help/documentation links

- Query/input panel with context-aware answer display
- Collapsible reference context
- Accuracy/comparison dashboard
- Admin metrics and usage panel
- Accessibility features: contrast, font size, dark/light mode toggle

4.2 Hardware Interfaces

The system is software-based and does not use special hardware. It runs on standard servers or cloud virtual machines with enough CPU or GPU power to handle inference tasks.

Recommended hardware:

Multi-core CPU (Intel i5, AMD Ryzen 5, or better)

16 GB RAM

Optional GPU (NVIDIA T4 or similar) for faster model execution

Client devices can include desktops, laptops, tablets, or smartphones, if they run a modern web browser.

4.3 Software Interfaces

- OpenAI API (openai>=1.0.0): For chat completions
- Sentence Transformers: For chunk embedding
- Cross-Encoder: For reranking
- Streamlit: Main app server and UI
- PDF/HTML parsing: pdfplumber, pypdf, beautifulsoup4, lxml
- REST API are exposed for internal modules, extensible for batch ops such as:
    - POST /route – handles routing
    - POST /answer – handles answer generation
    - GET /kb/{topic} – retrieves knowledge-based entries

- Metrics and output: JSON/CSV export as needed

All input and output data is in JSON format.

4.4 Communications Interfaces

- All external and internal connections use HTTPS (TLS 1.2+)
- Port configuration via environment variables or config files
- All API keys stored as environment variables
- Localhost or cloud-hosted Streamlit

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- ≤6 sec for API-hosted answers, ≤12 for local models (95% of cases)
- ≤2 sec for KB search
- ≥99% system uptime
- Support for 20+ concurrent users, with graceful error handling

5.2 Safety Requirements

- Does not provide offensive instructions; defense focused
- Filters and blocks unsafe/harmful queries
- Non-leaking error messages
- Licensed, ethical and compliant KB sources

5.3 Security Requirements

- API key stored securely in .env only
- All UI/admin data inputs validated

- No default passwords; secrets handled via env/libraries
- Role-based access (extensible for admin/dashboard views)
- All sensitive operations logged for auditing
- FERPA/GDPR compliance for educational/research use

5.4 Software Quality Attributes

- Reliability and crash recovery/integrity checks
- Usability for all skill levels (beginner-expert)
- Modular maintainable codebase (retriever, reranker, UI, metrics)
- Portability: runs on all major platforms, easy to Dockerize
- Scalability for growing knowledge bases and user needs
- Extensibility for new models, formats, or KB sources
- Bias reduction, fair access, and full content transparency

6. Other Requirements

Regulatory Compliance – If used in schools or universities, the system shall follow institutional rules on student privacy and data protection.

Internationalization – Future releases may add multi-language support. The design shall allow new language models and localized content.

Sustainability – The system shall track energy use of hosted models. Local models shall apply quantization and use efficient hardware to lower environmental impact.

Community Contributions – The system shall allow outside contributors to suggest new knowledge-based entries through a controlled review process.

Appendix A: Glossary

API – Application Programming Interface.

Embedding – A vector form of text used for semantic search.

Fishbone (Ishikawa) diagram – Visual tool for grouping causes or topics; here used to organize cybersecurity knowledge.

Knowledge base (KB) – A structured store of cybersecurity information used for retrieval and context.

Large language model (LLM) – A neural network trained on large text datasets that can generate human-like responses.

Prompt – Structured input that guides an LLM to create a response.

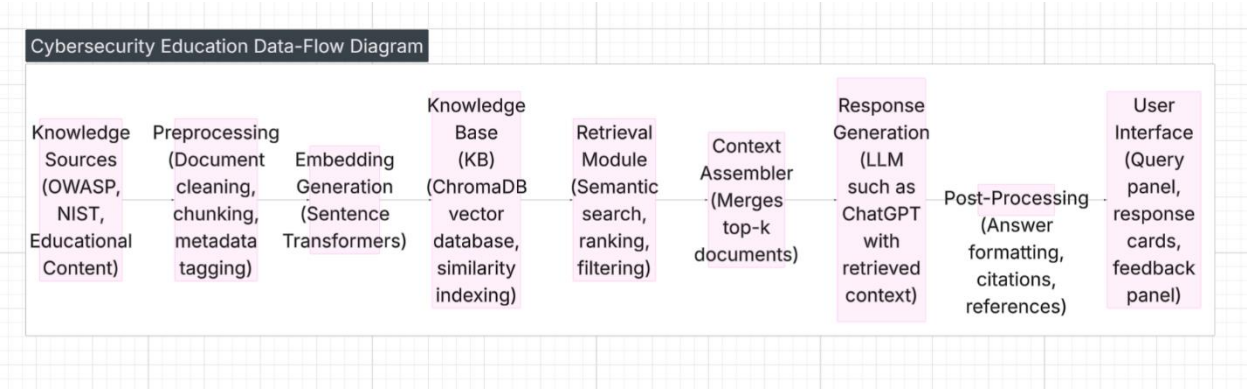
Retrieval-Augmented Generation (RAG) – A method that combines retrieval of facts with a generative model to improve accuracy.

Vector database – A database built to store and search high-dimensional vectors.

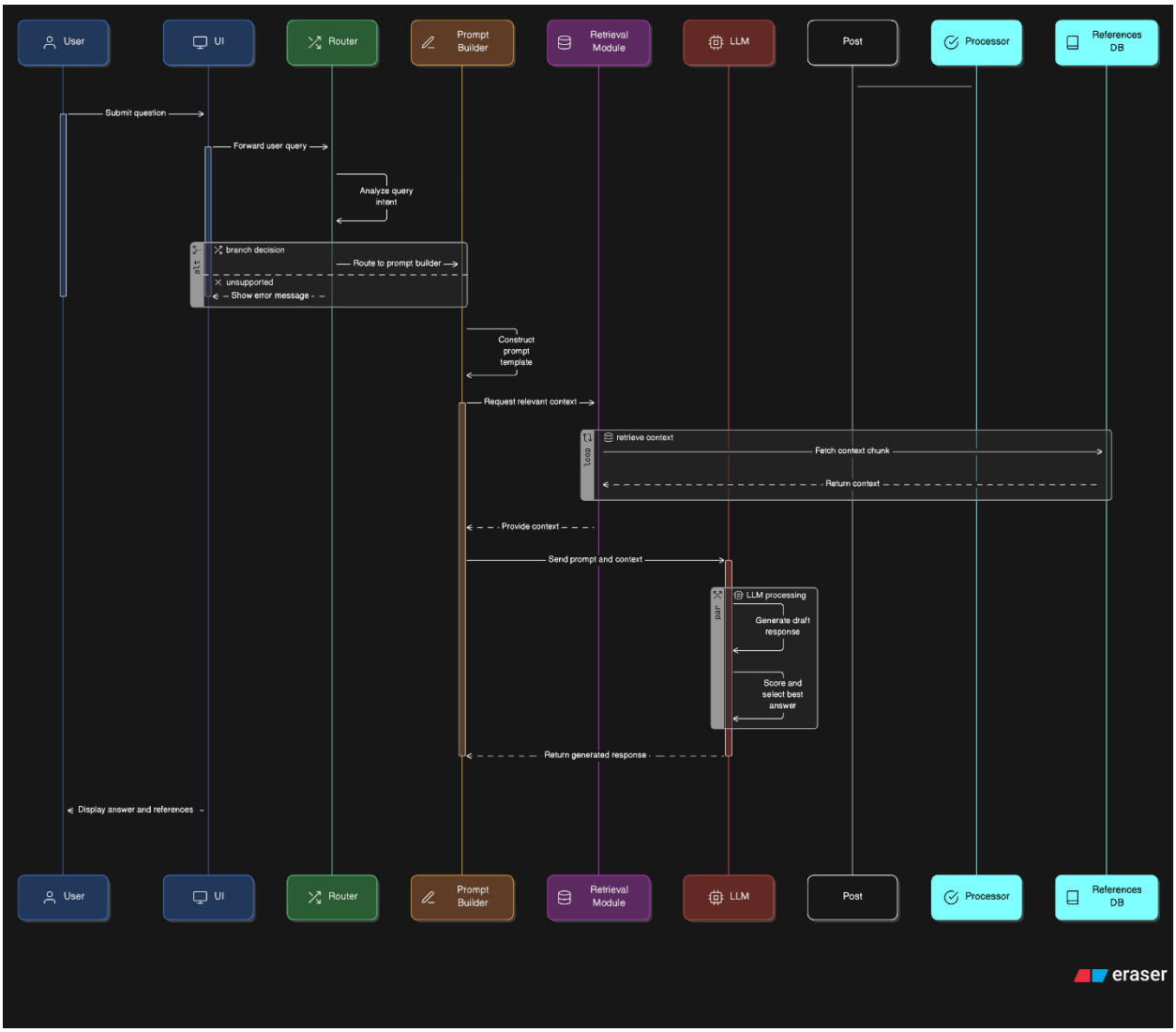
Zero Trust Architecture – A security model based on "never trust, always verify."

Appendix B: Analysis Models

Data-Flow Diagram – Shows how data moves from knowledge sources into the KB, through embedding generation, retrieval and response-generation stages.



Sequence Diagram – Illustrates the typical interaction sequence: user submits query → router determines branch → prompt builder creates prompt → retrieval module gathers context → LLM generates response → post-processor adds references → UI displays answer.



Appendix C: Issues List

Model Bias – Ongoing checks are needed to spot and reduce bias in generated responses.

Reference Availability – Some outside sources may change or vanish. The system shall use link checking and archival storage.

Scaling Knowledge Base – If the KB grows beyond 10,000 documents, the system may need distributed vector databases.

Browser Compatibility – The web UI must be tested on many browsers and devices. Any speed or rendering issues shall be fixed.

User Privacy – Even if logs are anonymized, legal experts should review data collection to ensure it follows privacy rules.

History:

VERSION	DATE
1	09/08/2025
2	09/25/2025
3	10/07/2025
4	11/17/2025